Data Structures and Algorithms

Lecture 14

Hash table and hash function

Dept. of ICT, Faculty of Technology- UoR

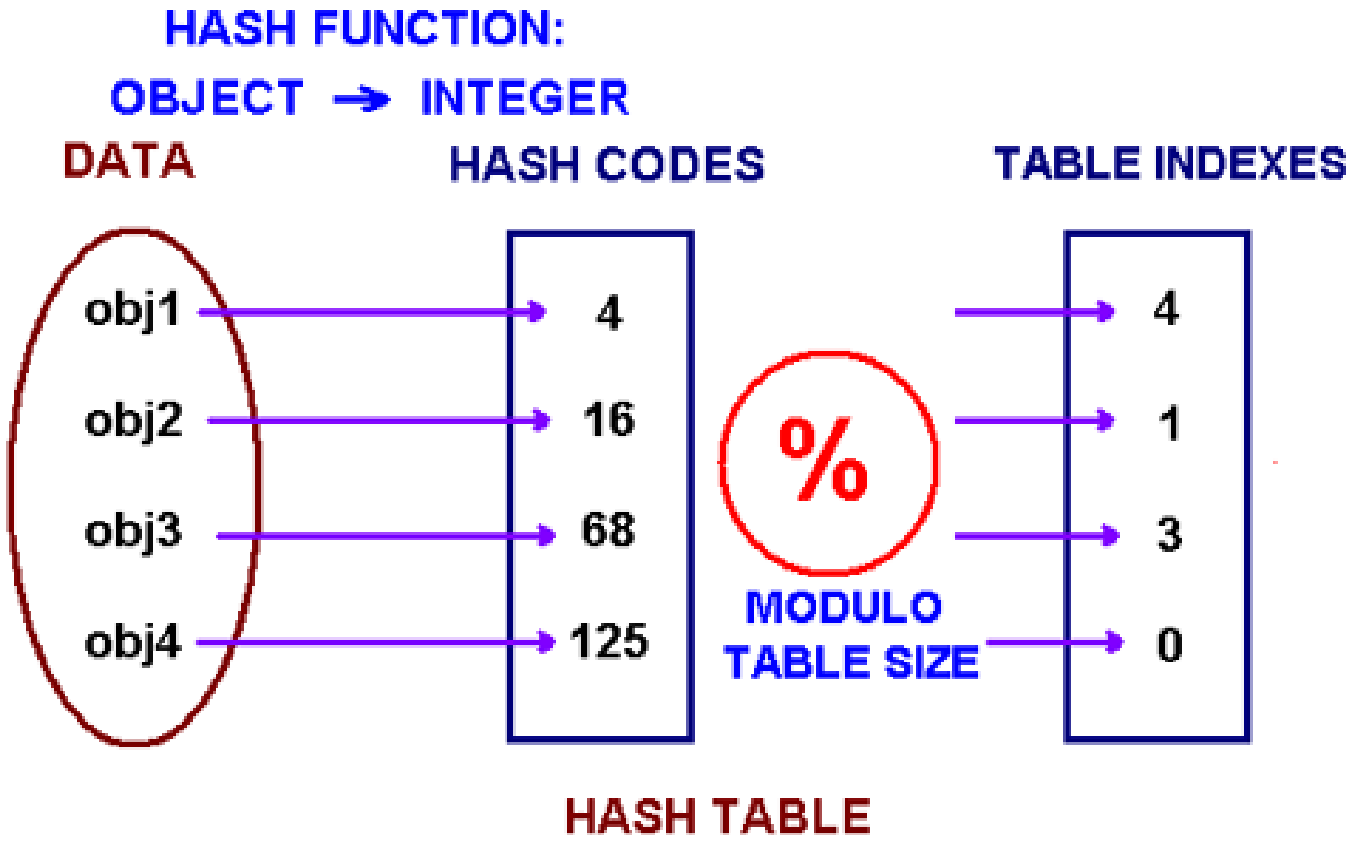
Objectives

- to introduce concept of “hash-index”.

Concept of Hashing

- The problem at hands is to speed up searching……..
- Consider the problem of searching an array for a given value.
- If the array is not sorted, the search might require examining each and all elements of the array.
- We could search even faster if we know in advance the index at which that value is located in the array.
- Suppose we do have that magic function that would tell us the index for a given value.
- With this magic function our search is reduced to just one probe, giving us a constant runtime O(1).
- Such a function is called a **hash function** . A hash function is a function which when given a key, generates an address in the table.

Concept of Hashing



HASH TABLE

OBJ4	OBJ2		OBJ3	OBJ1
0	1	2	3	4

Introduction

- **Hash tables** are a common data structure.
- They consist of an array (the **hash table**) and a mapping (the **hash function**).
- The hash function maps **keys** into **hash values**. Items stored in a hash table must have keys.
- The hash function maps the key of an item to a hash value, and that hash value is used as an index into the hash table for that item.
- This allows items to be inserted and located quickly.

Introduction

Definition

Hashing is the process of mapping a key value to a position in a table.

A **hash function** maps key values to positions.

A **hash table** is an array that holds the records.

Searching in a hash table can be done in O(1) regardless of the hash table size.

Introduction

- Very VERY fast way to build and access tables (and records in files too).
- Provide nearly O(1) performance.
- Simple to do too.
- Extremely fast; significantly faster than trees, which are $O(\log_2 n)$!

More…

- Similar to arrays – which is a disadvantage.
- **Do need** to have some idea how many items will populate the hash table so table size can be determined.
- Also quite difficult (makes no sense) to access sequentially (as you will see).
- But you will know the size of the table and hence the size of your ‘database.’

Introduction to Hashing

- Often called **Randomizing**…
- We will have a range of (input) values (e.g. state names, account numbers, String value, etc.) that will be **mapped** into a **range** of **array values**.
- The **function** that will map the key values of these records, objects, etc. to table locations (or relative file locations) is called a **hash function.**
- **So, items (objects, records, attribute, etc.) will 'hash' to locations in the hash table based on a value within a record / object.**
- **Then, the record / object will be moved into and occupy those locations**.

Introduction to Hashing - Example

- Let's take objects of type Person that have an SSAN attribute (field) in it.

- We may take the last four digits, like 1234, and divide (mod) that by some prime number, say 47, such that ALL objects will hash to values between, say, 0 and 46.

- So, given an input object, we will access this key value, **hash to an index** between 0 and 46 and then move the entire object (or whatever it is…) into this location (indexed by the hashed-to value (0 to 46).

Hashing: Exception Cases

- **Existing Key:** Sometimes there is an attribute **within an object** that can be **directly used** as an index into the hash table without invoking a hashing function.

- **Examples**: e.g. (from your book) employee number in an object or record, or, in a laboratory, an 'experiment' number attribute, which might start with a lab number or experiment number 0 or 1.

- In these cases, the index number into the hash table is the same as the key value, and we don't have to actually 'hash' to a table value.

- We can use this integer as the index into the table we are building.

Hashing: Converting Words to Numbers

- Our input into a hashing algorithm may sometimes be characters or a String. So we may want to convert them to numeric form to facilitate randomizing.
- One way to convert letters to numbers is to translate:
  - a = 1; b = 2, t = 20, s = 19. Add the total for a word such as cats, which yields a 43.
  - So 'cats' would be stored in location 43.
- Unfortunately, it may be that many words might 'hash' to 43. These are called '**synonyms**.'
- But **synonyms** cannot all occupy the same single space…So what to do? (Coming soon!)

Introduction Hashing Algorithms

Given a key (attribute) in an object (or record), we **need a hash function** that will map **all** possible keys in the **input space** into a specific location within the **bounds** of the hash table.

What we'd like is to **hash** to a table location and discover this location is **unoccupied** and then **move** the object into the table at the hashed-to location.

It just doesn't always work that way in practice.

Introduction Hashing Algorithms (Collisions and Synonyms – Terminology)

- When one or more keys hash to the same hash table location, we call this situation a **collision.**

- Those elements (objects, records, …) that do hash to this location are called **synonyms.**

- So, we need a mechanism (algorithm) to deal with synonyms.

Introduction Hashing Algorithms

- So, practical considerations require us to map key values (including synonyms) into hash table locations that may NOT be one-to-one.

- Also, **all input values** themselves must map to the **range of hash table entries** (the table size) either directly or via a hashing algorithm.

- Thus, we may have something like:
- **Arrayindex = someNumber % arraySize or**

  Arrayindex = someKeyConvertedToANumber % arraySize

Introduction Hashing Algorithms

Arrayindex = someNumber % arraySize

➔ classic example of a hashing function! Called Division-Remainder Method.

This algorithm hashes (converts) a 'number' in a **large** range (naturally occurring values – say social security numbers) into a number in a **much smaller** range (the table size).

- (As one might expect, there is a very large number of hashing functions available. (See Donald Knuth))

Hashing and Collision Example

- So, we have:

  Arrayindex = someNumber % arraySize

So, any **integer mod arraySize** results in an Arrayindex range of 0 to arraySize-1.

5280 (number of feet in a mile) mod 51 yields a number between 0 (if 51 divides 5280 evenly) and 50.

Any number mod 51 yields a number between 0 and 50. Period!

Again, this particular hashing algorithm is called the **Division Remainder method!**

Consider the following exercise:

Hashing and Collision Example

- Clearly, we do not have space for all naturally-occurring numbers uniquely. Consider an attribute:

1. $\frac{42000}{125}$ **= QUOT = 16, REM=0 => index = 0**

2. $\frac{42200}{125}$ **= QUOT = 17, REM=75 => index = 75**

3. $\frac{42200}{125}$ **= QUOT = 17, REM=95 => index = 95**

4. $\frac{42249}{}$ **= QUOT = 17, REM=124 => index = 124**

LOOKS GOOD...

ALL KEY-TO-ADDRESSS TRANSFORMATIONS SEEM TO HASH TO ADDRESSES BETWEEN 1 AND 125.

Actually, we 'forced' all input attributes to hash into this range.

Hashing and Collision Example

BUT: CONSIDER THE KEYS: 42125, 42250, 42375, ...

ALL HASH TO THE SAME ADDRESS!!!

- And 40,001, 42,126, 42,151 ALL HASH TO 1 THAT IS, REL KEY = 1 or index = 1.

- THUS WE WILL HAVE / may have MANY keys that will hash to the same address (location) in the hash table.

- Those input integers (integers in this case) are examples of synonyms!

Hashing and Collision Example

- This **oftentimes** arises from a large range of input numbers mapping into much smaller target array sizes.

- This is the price we pay to save space. ➔
- We cannot reserve a ‘spot’ for every possible value that just ‘might’ hash to a value in a large range.
- Our table would have to be much too large to be practical.

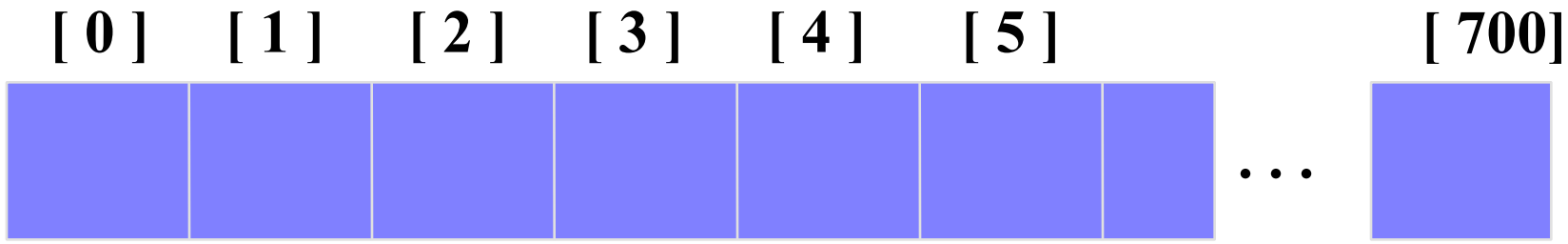
- There is no guarantee a key will not hash to the same location occupied by another object.

- As stated, when two or more keys hash to the same location, we have a **collision.**

- **So, what do we do? Only one object can occupy a single location in the hash table.**

What is a Hash Table ?

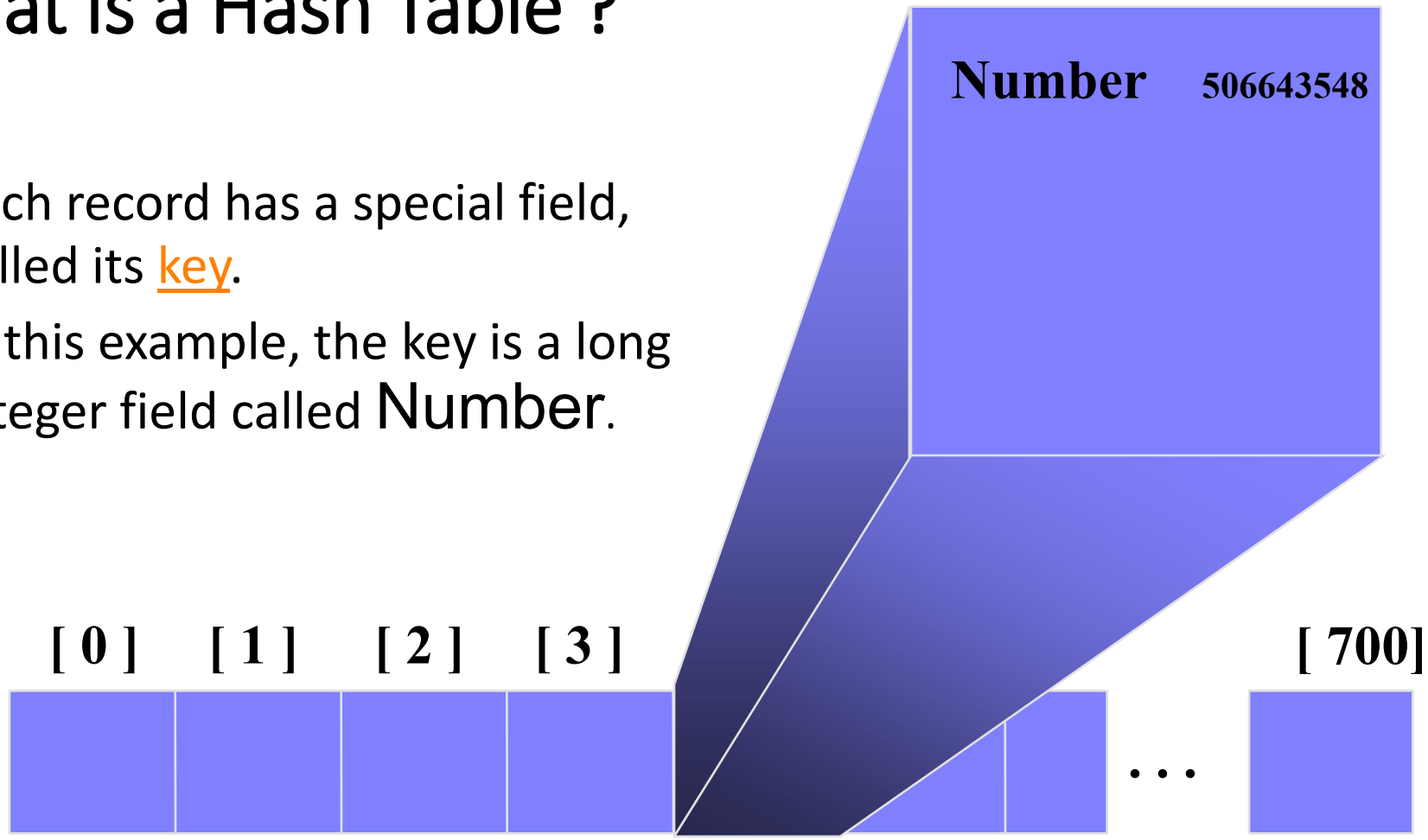
- The simplest kind of hash table is an array of records.
- This example has 701 records.



An array of records

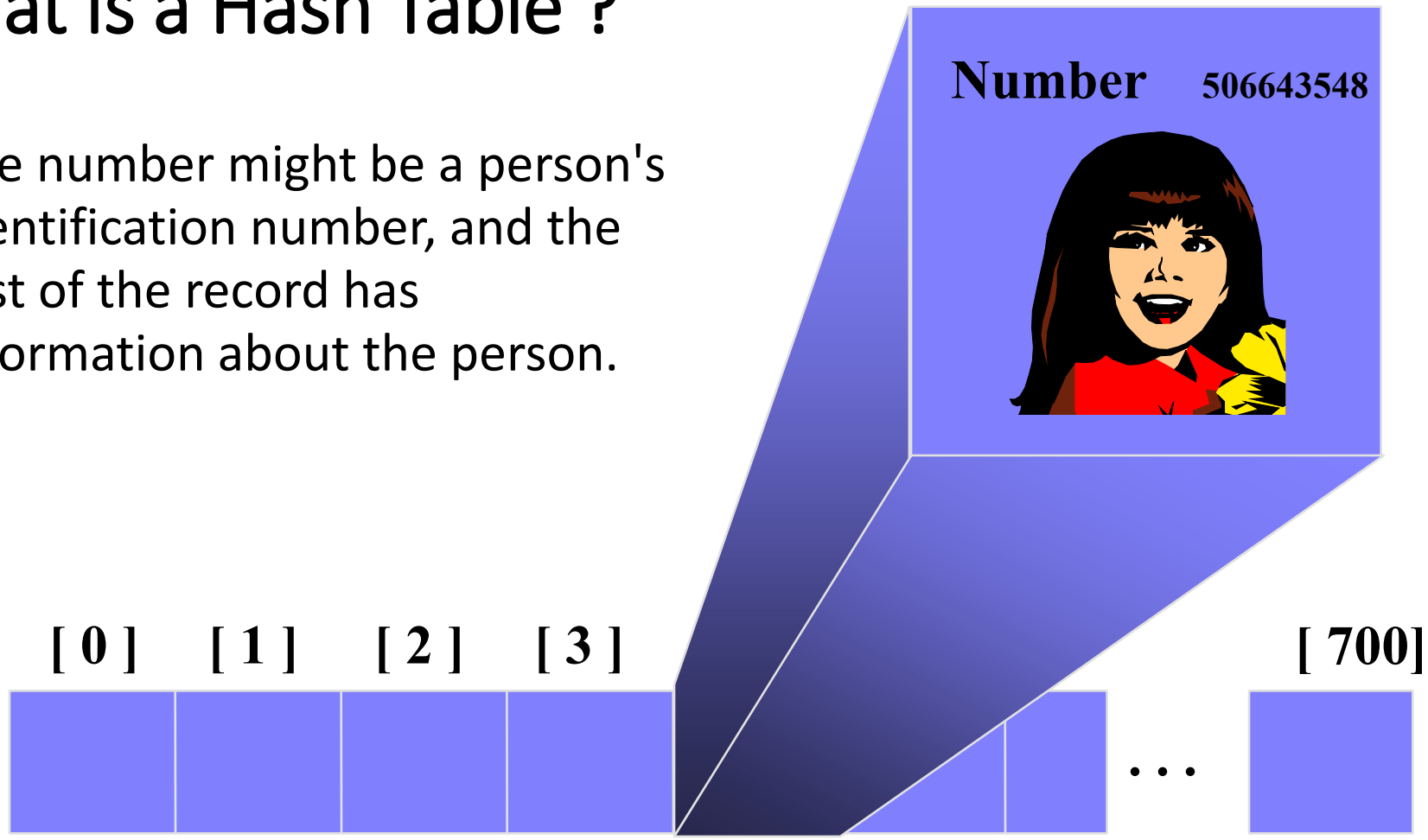
What is a Hash Table ?

- Each record has a special field, called its key.
- In this example, the key is a long integer field called Number.



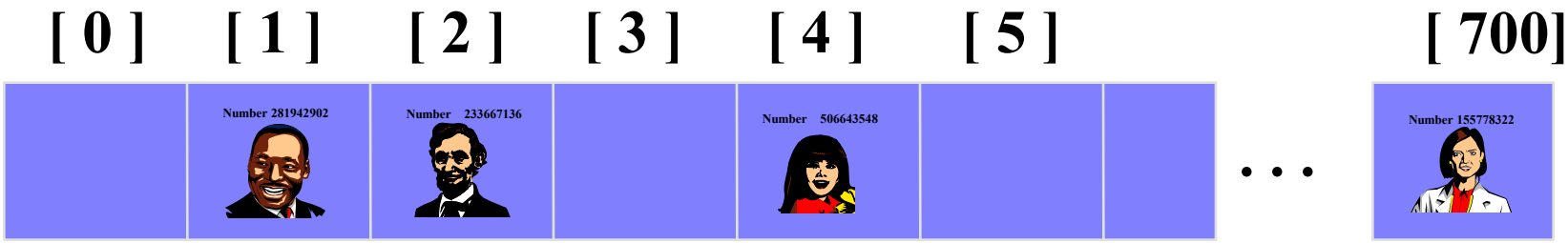
What is a Hash Table ?

- The number might be a person's identification number, and the rest of the record has information about the person.



What is a Hash Table ?

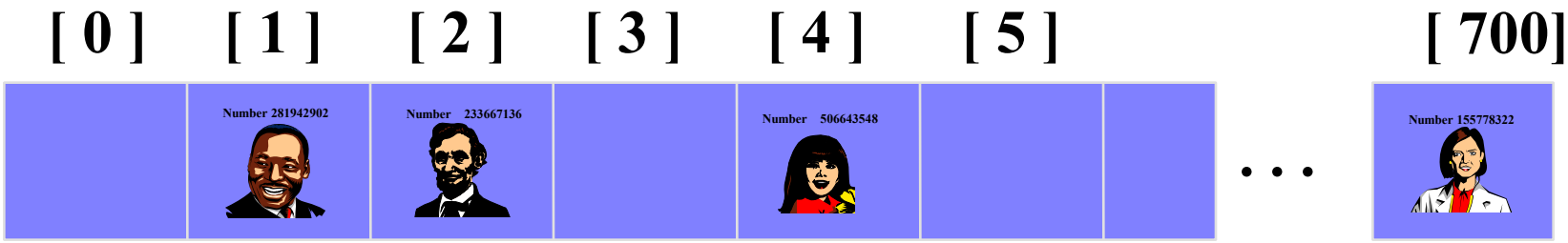
- When a hash table is in use, some spots contain valid records, and other spots are "empty".



Inserting a New Record



- In order to insert a new record, the **key** must somehow be **converted to** an array **index**.
- The index is called the **hash value** of the key.

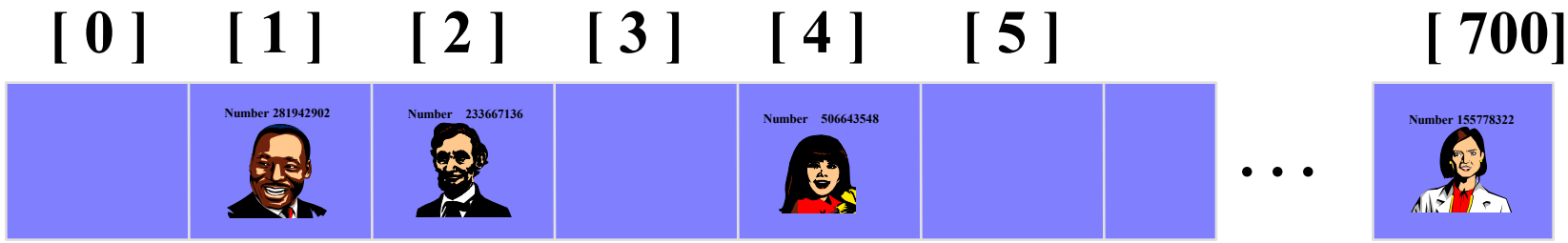


Inserting a New Record



- Typical way create a hash value:

What is (580625685 mod 701) ?

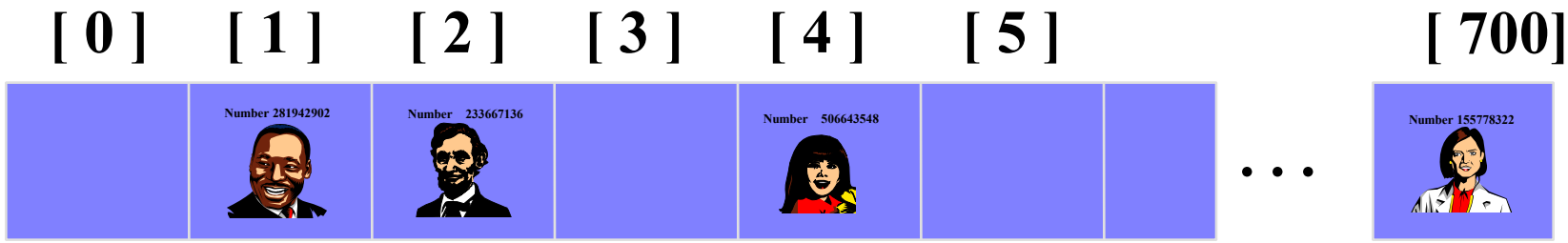


Inserting a New Record

- Typical way to create a hash value:

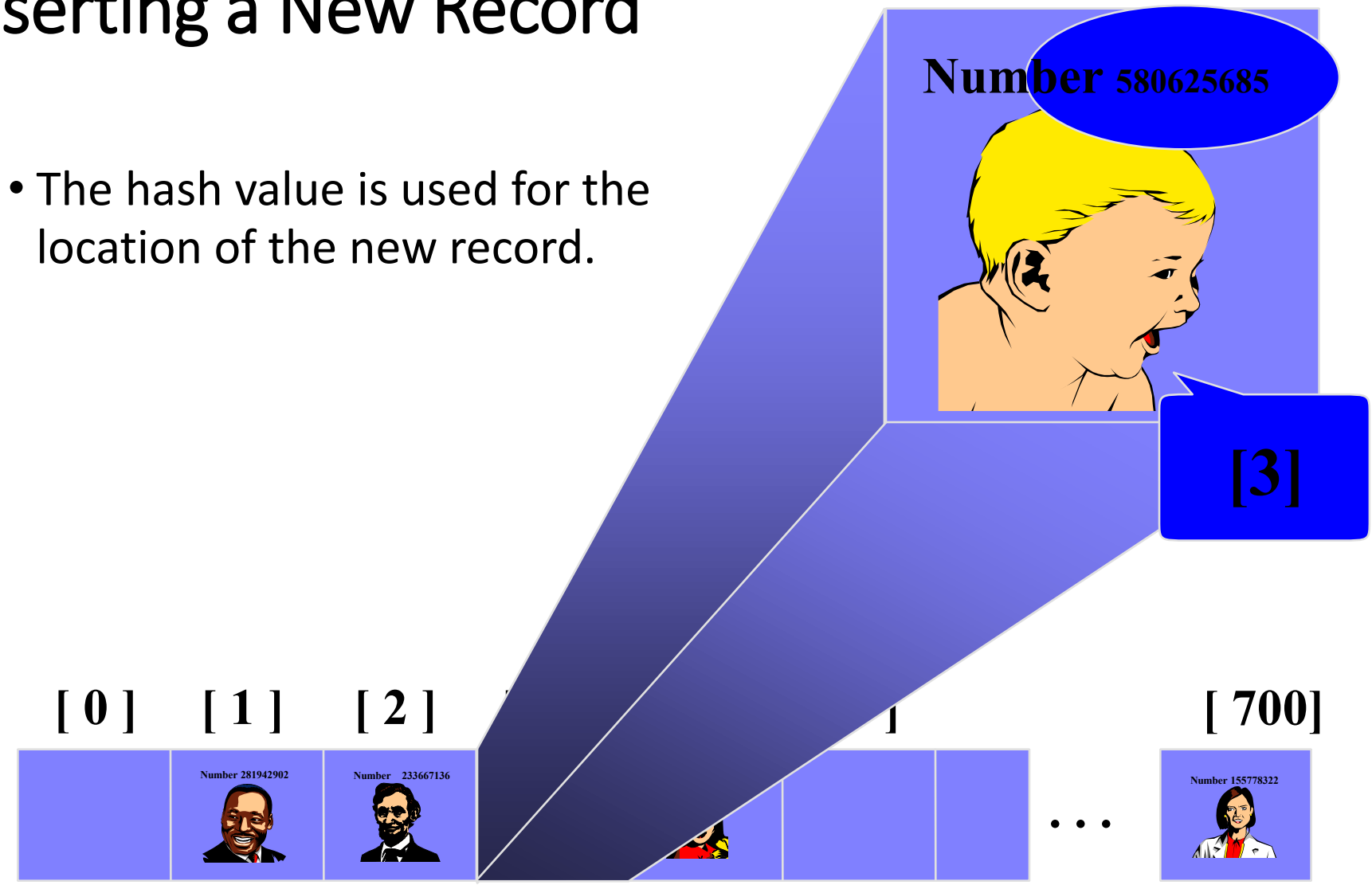


What is (580625685 mod 701) ?



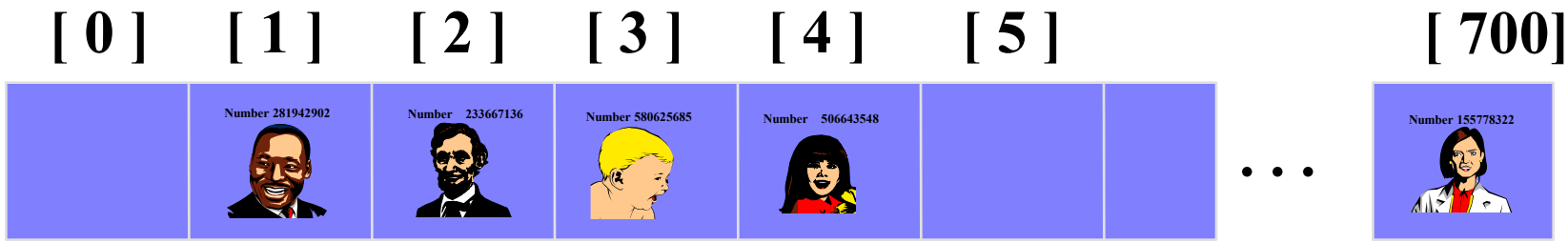
Inserting a New Record

- The hash value is used for the location of the new record.



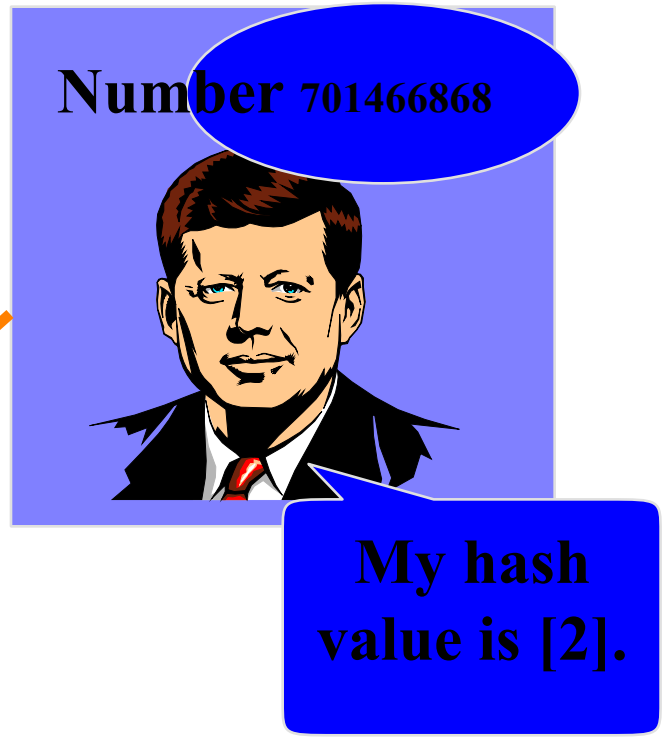
Inserting a New Record

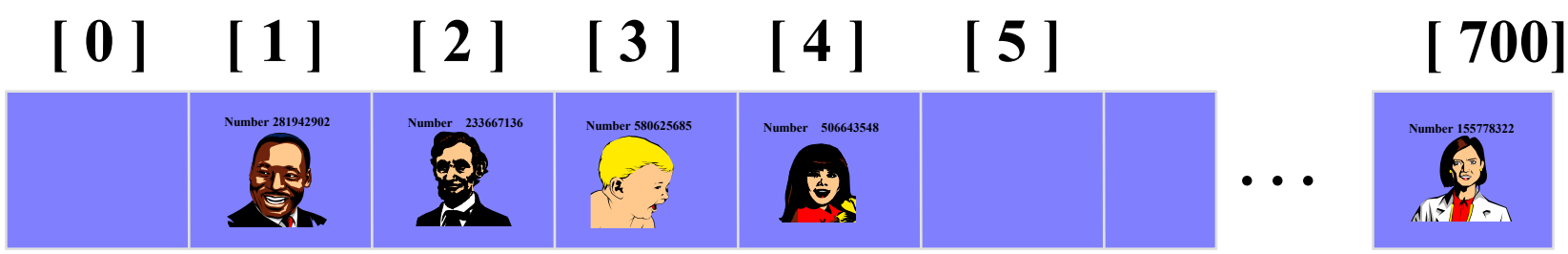
- The hash value is used for the location of the new record.



Collisions

- Here is another new record to insert, with a hash value of 2.



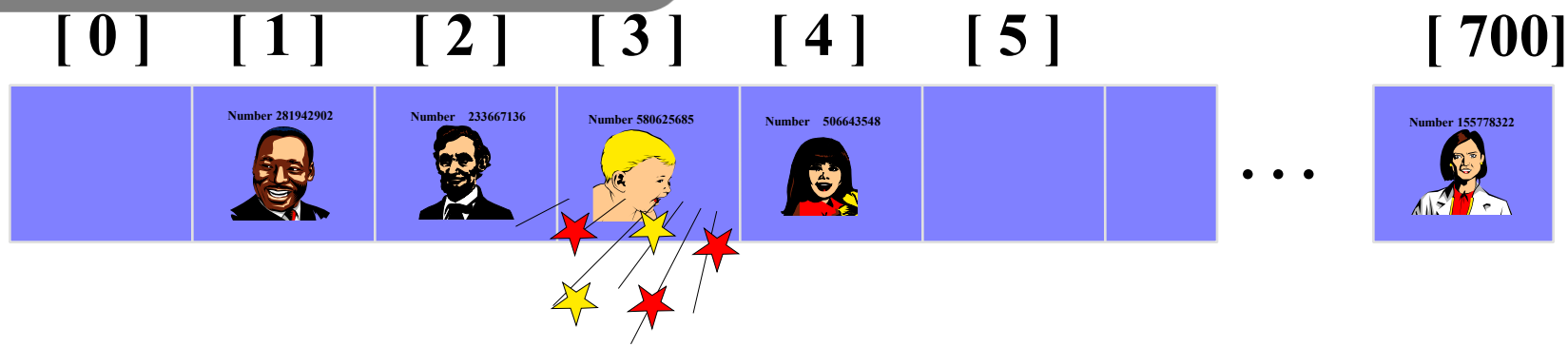


Collisions

- This is called a **collision**, because there is already another valid record at [2].



When a collision occurs, move forward until you find an empty spot.

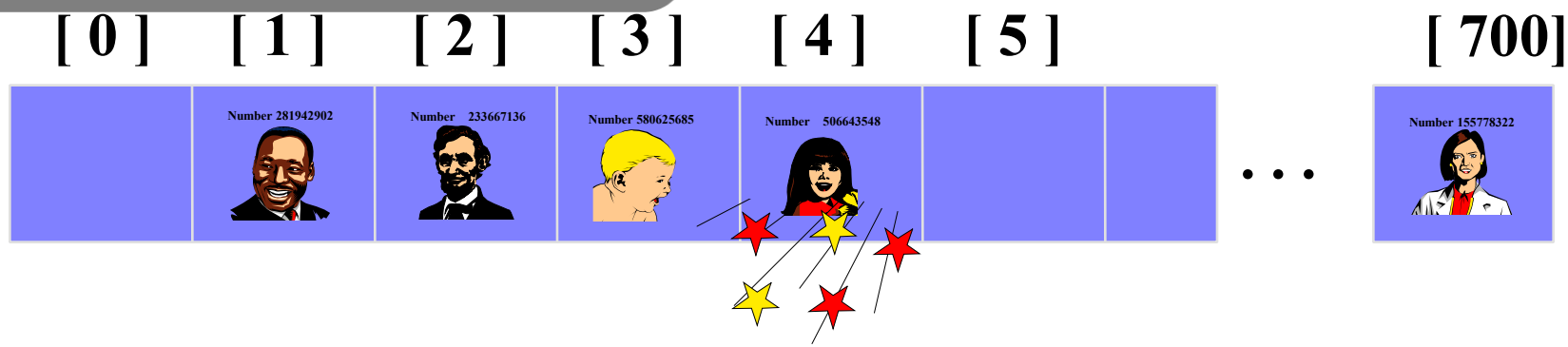


Collisions

- This is called a **collision**, because there is already another valid record at [2].



When a collision occurs, move forward until you find an empty spot.

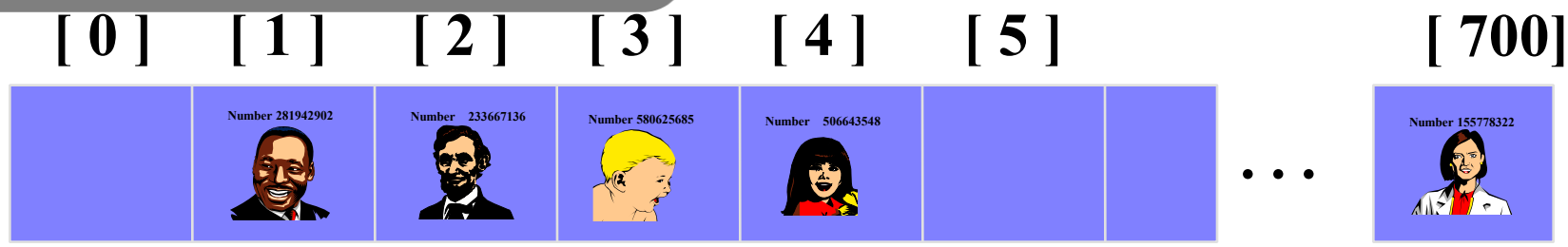


Collisions

- This is called a **collision**, because there is already another valid record at [2].



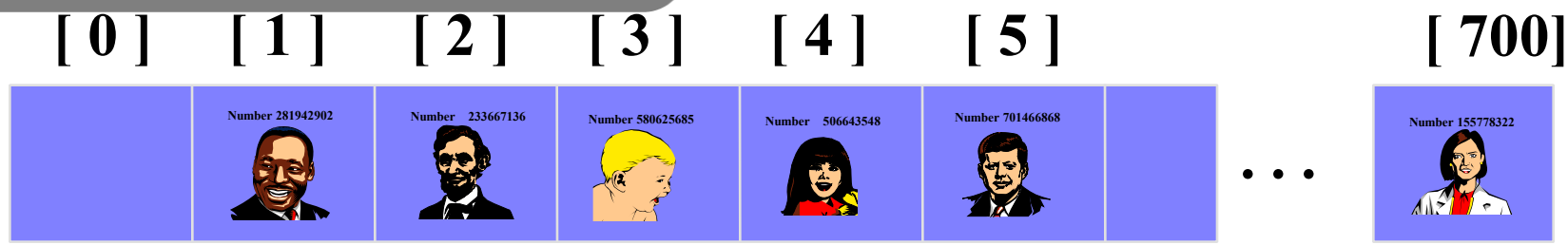
**When a collision occurs,
move forward until you find an empty spot.**



Collisions

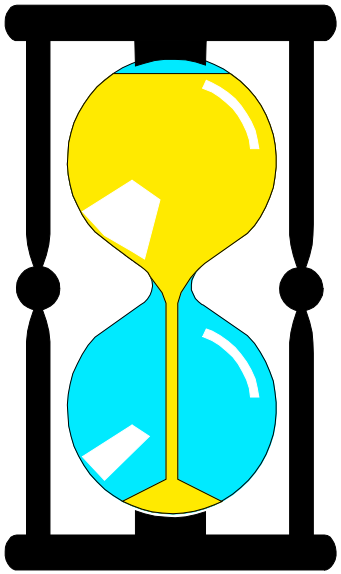
- This is called a **collision**, because there is already another valid record at [2].

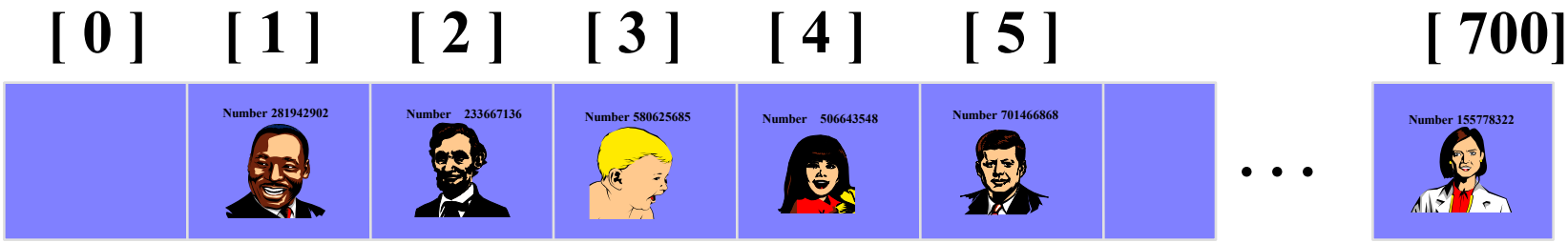
The new record goes in the empty spot.



A Quiz

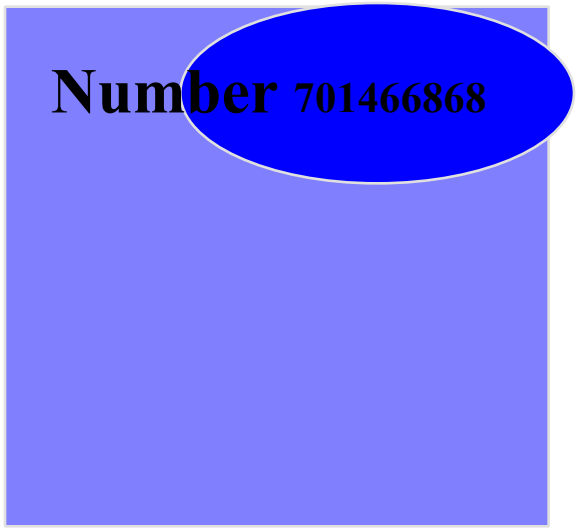
Where would you be placed in this table, if there is no collision? Use your social security number or some other favourite number.

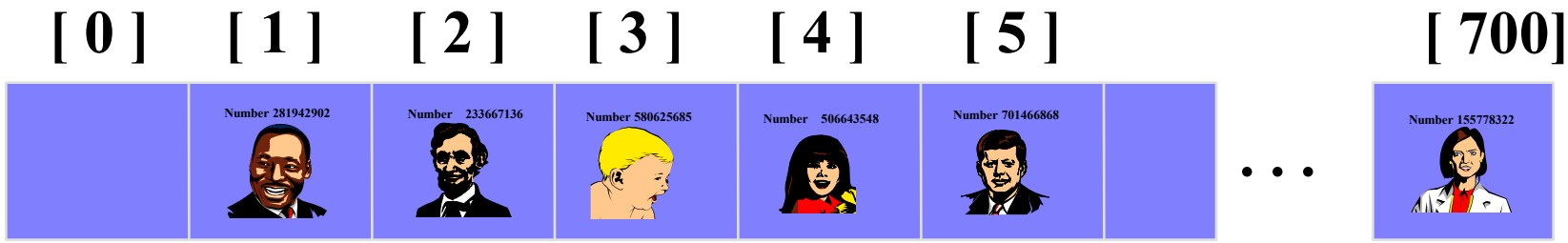




Searching for a Key

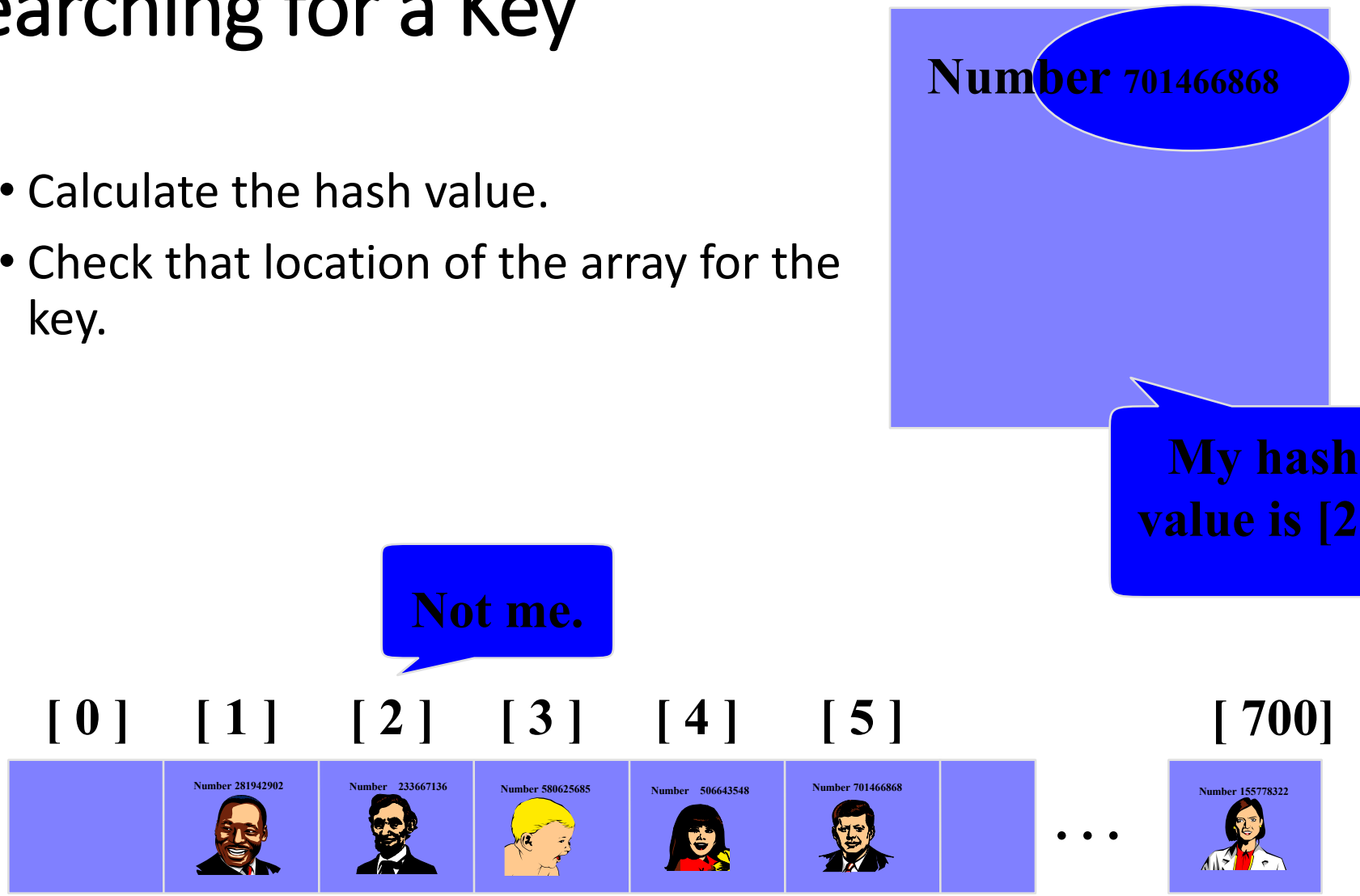
- The data that's attached to a key can be found fairly quickly.





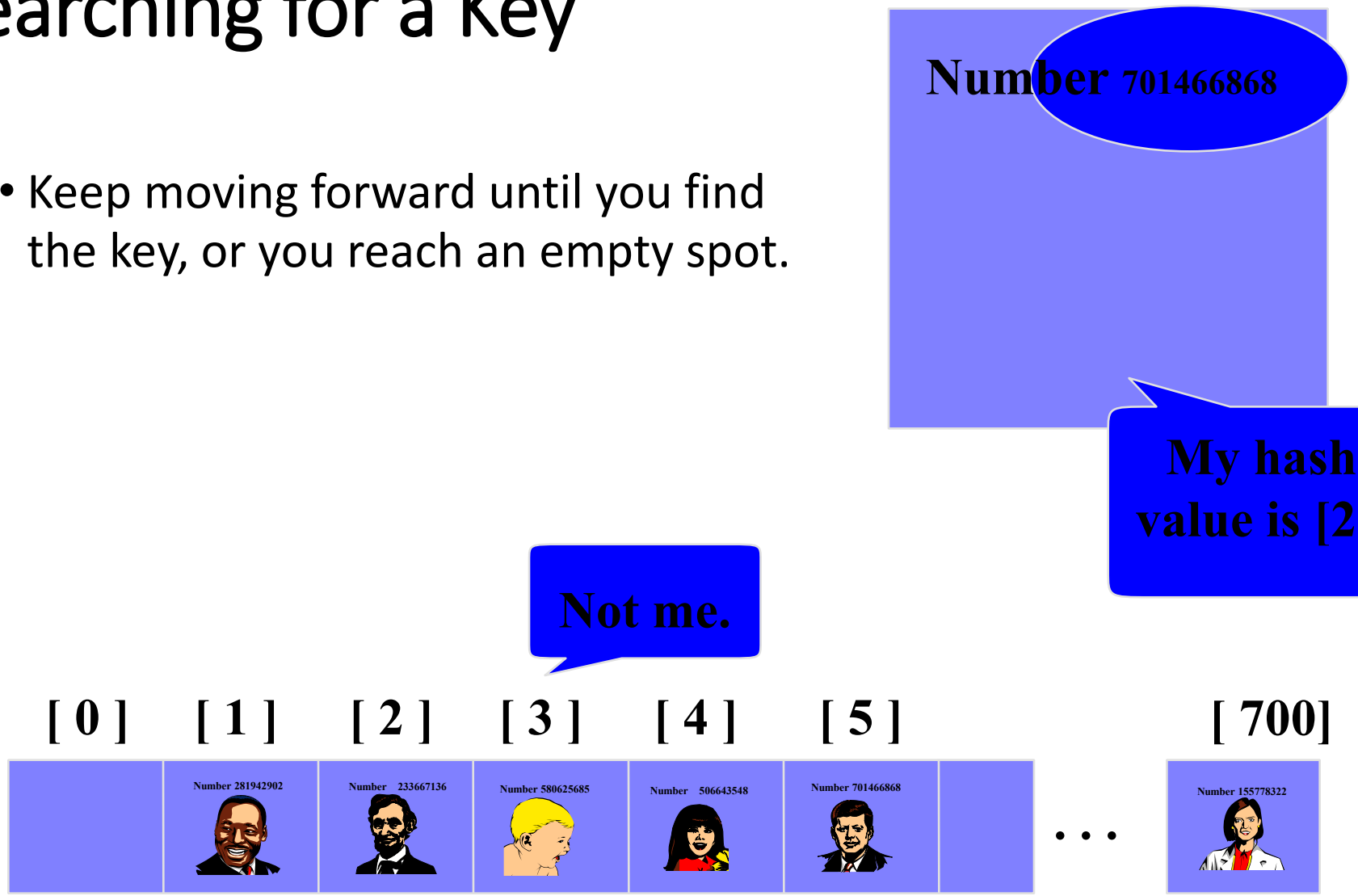
Searching for a Key

- Calculate the hash value.
- Check that location of the array for the key.



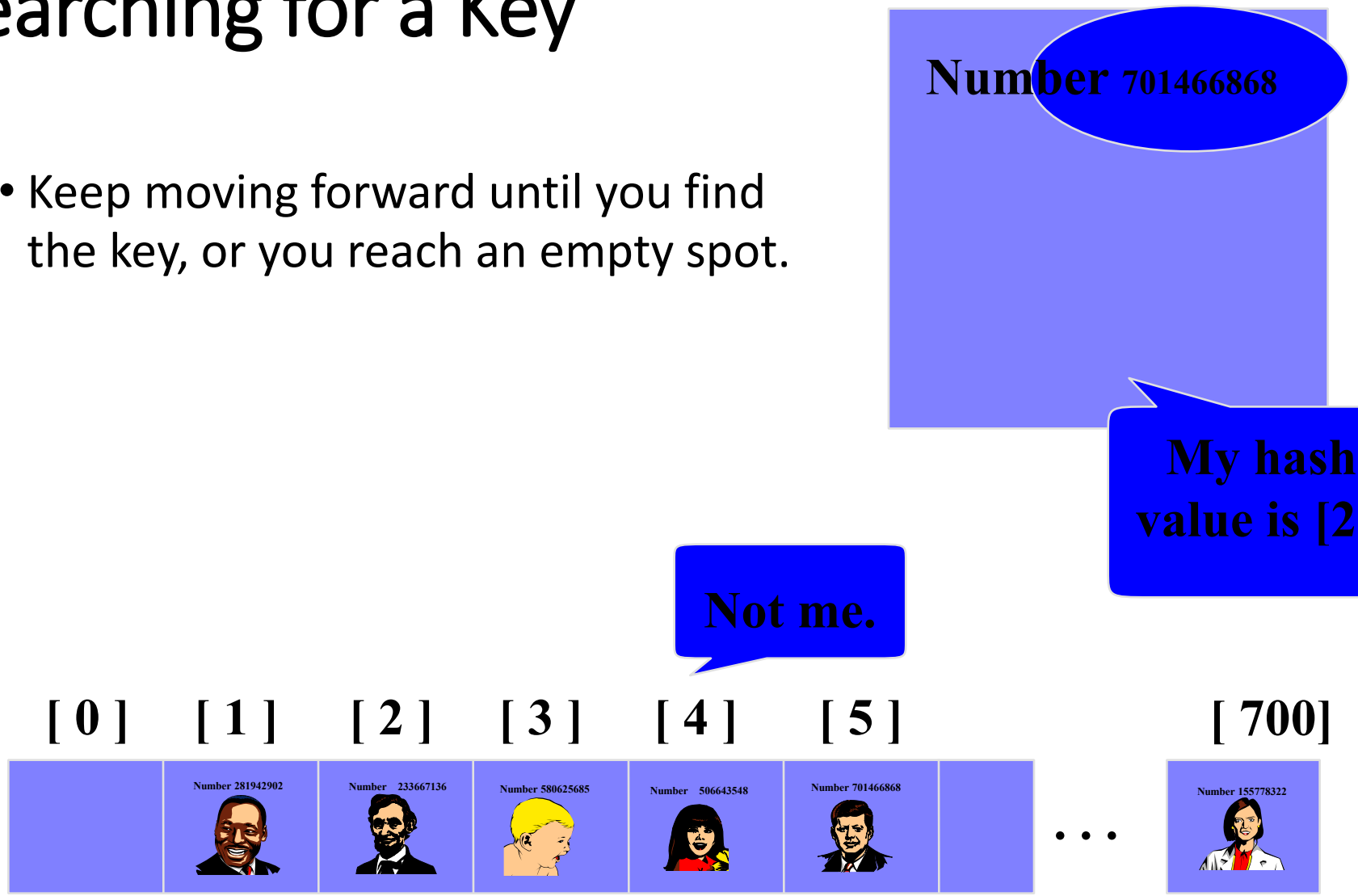
Searching for a Key

- Keep moving forward until you find the key, or you reach an empty spot.



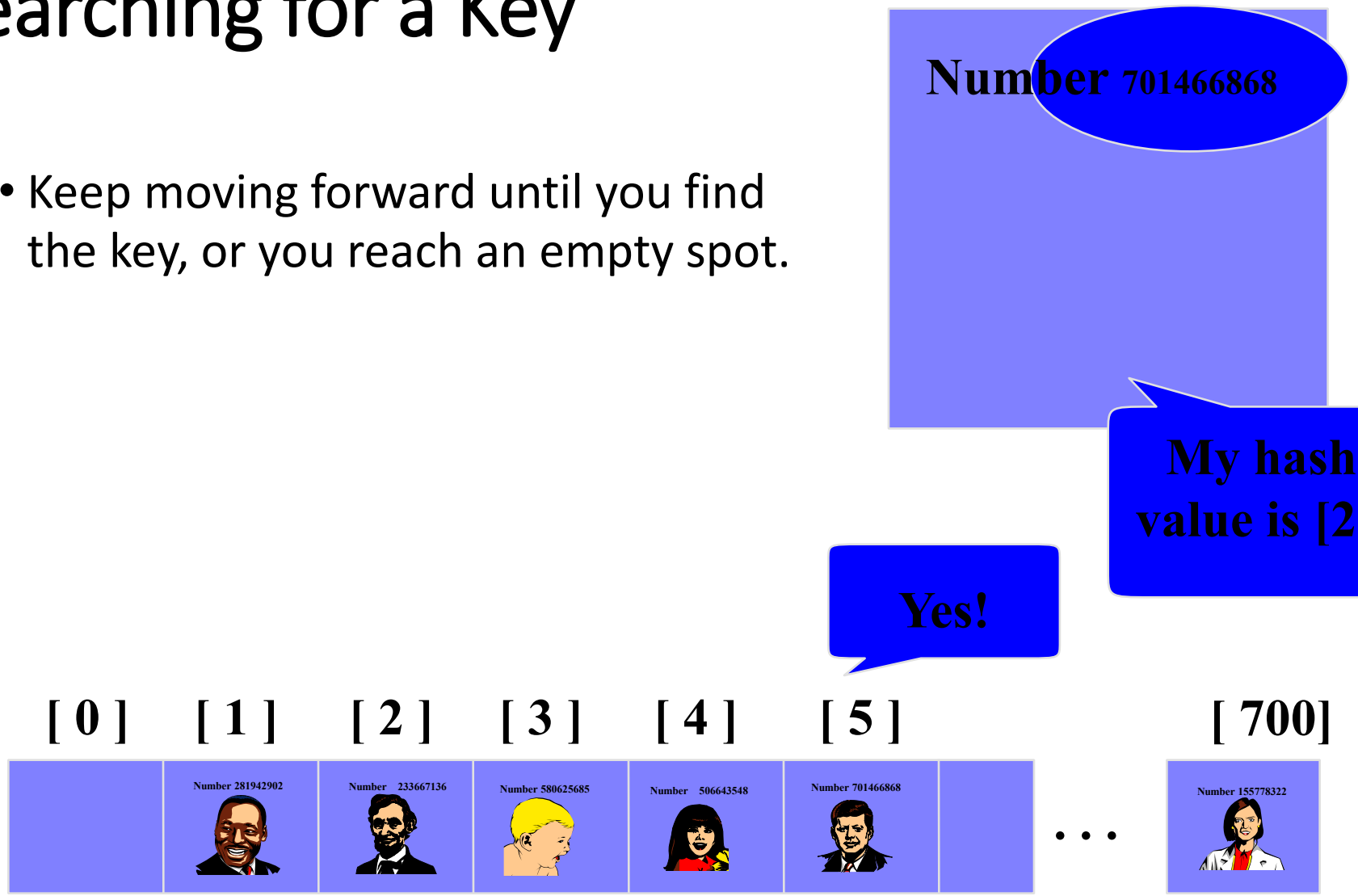
Searching for a Key

- Keep moving forward until you find the key, or you reach an empty spot.



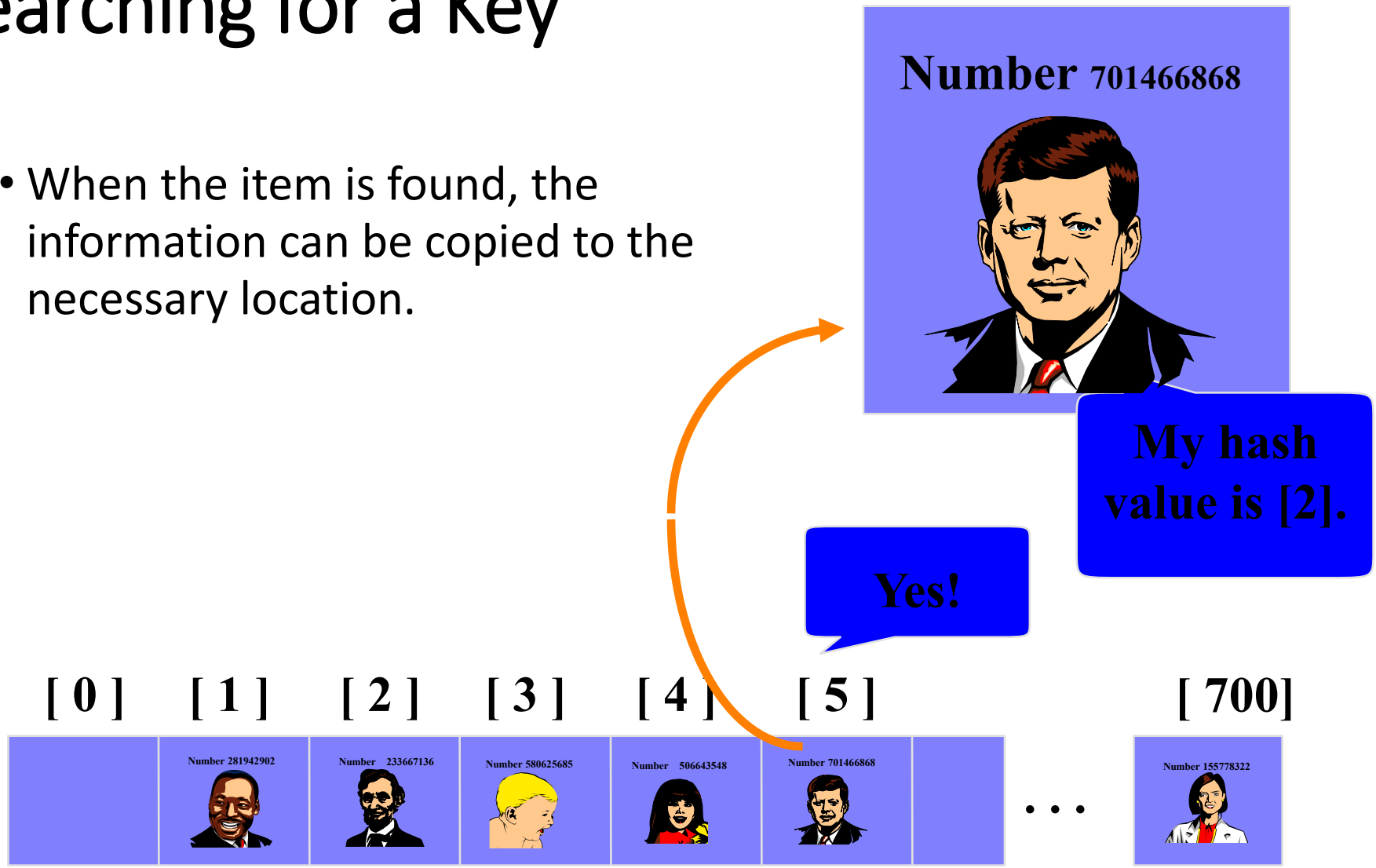
Searching for a Key

- Keep moving forward until you find the key, or you reach an empty spot.



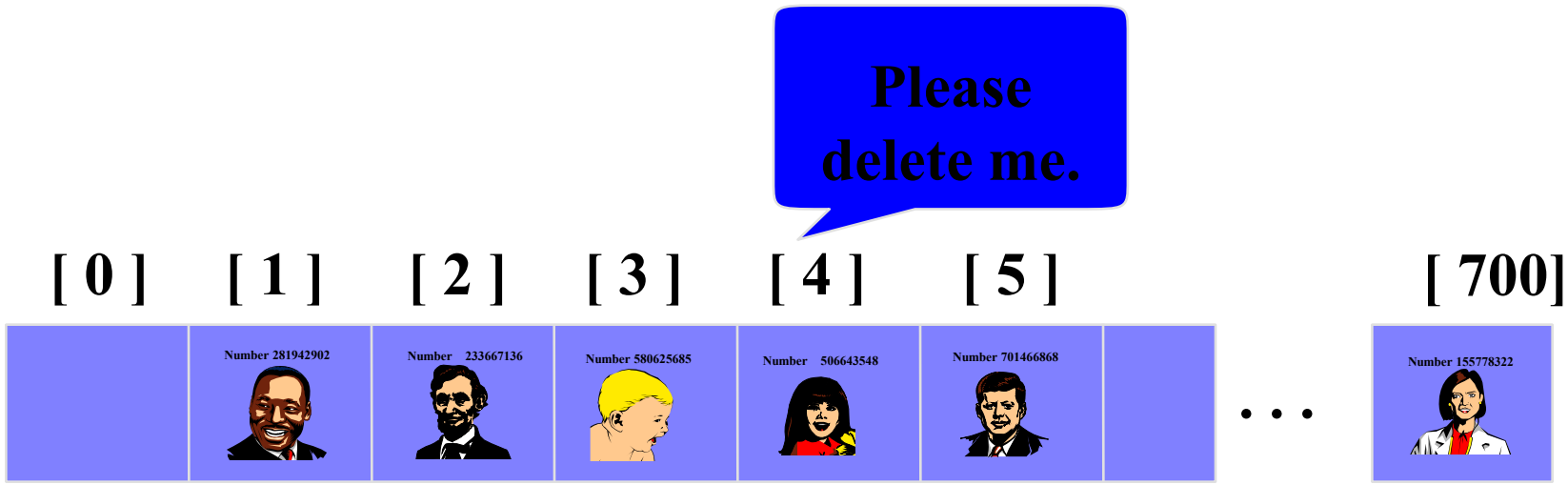
Searching for a Key

- When the item is found, the information can be copied to the necessary location.



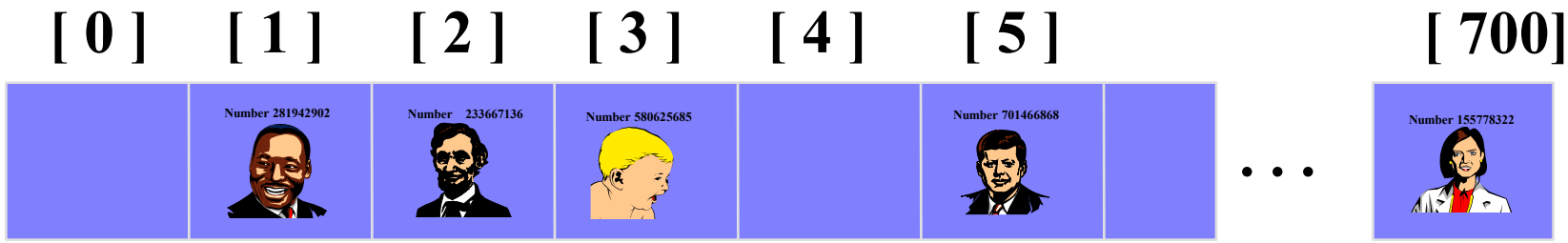
Deleting a Record

- Records may also be deleted from a hash table.



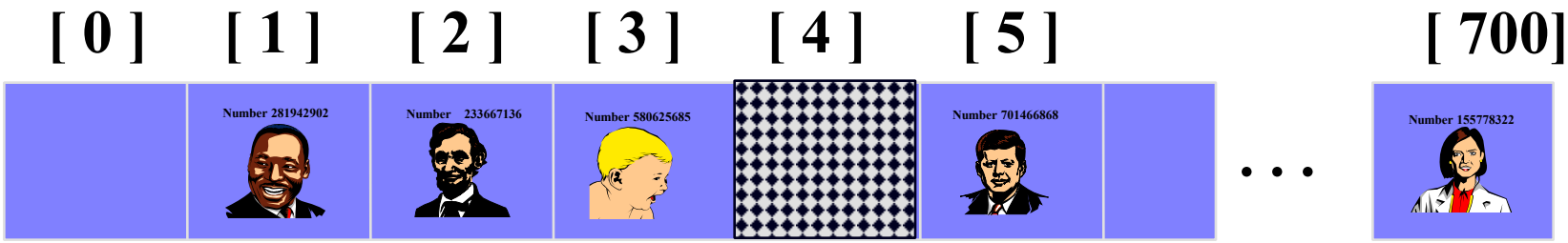
Deleting a Record

- Records may also be deleted from a hash table.
- But the location must not be left as an ordinary "empty spot" since that could interfere with searches.



Deleting a Record

- Records may also be deleted from a hash table.
- But the location must not be left as an ordinary "empty spot" since that could interfere with searches.
- The location must be marked in some special way so that a search can tell that the spot used to have something in it.



Hash Function

What is a Hash Function ?

- Hash functions are extremely useful and appear in almost all information security applications.

- A hash function is a mathematical function that converts a numerical input value into another compressed numerical value. The input to the hash function is of arbitrary length but output is always of fixed length.

- Values returned by a hash function are called message **digest** or simply **hash values**.

Main features of Hash function

Fixed Length Output (Hash Value)

- Hash function coverts data of arbitrary length to a fixed length. This process is often referred to as hashing the data.

- In general, the hash is much smaller than the input data, hence hash functions are sometimes called compression functions.

- Since a hash is a smaller representation of a larger data, it is also referred to as a digest.

Main features of Hash function

Efficiency of Operation

- Generally, for any hash function h with input x, computation of h(x) is a fast operation.

- Computationally hash functions are much faster than a symmetric encryption.

END